# ros2-amr-fleet-guard

Multi-AMR (Autonomous Mobile Robots) warehouse simulation and fleet-management project built around ROS 2 Jazzy, Gazebo, Nav2, AMCL, and a custom fleet traffic coordinator.

The project currently supports up to 8 namespaced AMRs, autonomous pick/drop missions, battery-aware task allocation, automatic dock return/charging, fleet monitoring, and a central traffic manager that routes robots through a generated warehouse lane graph.

> **Current development status:** the traffic manager significantly reduces head-on conflicts by reserving lane resources, but it is intentionally conservative: it reserves all constrained resources for an entire navigation leg. This improves safety at the cost of fleet throughput, and some head-on/conflict cases can still occur. Rolling reservations and direction-aware traffic is planned next.

## Current baseline

This README reflects `main` at commit `51b827b` (2026-11-04).

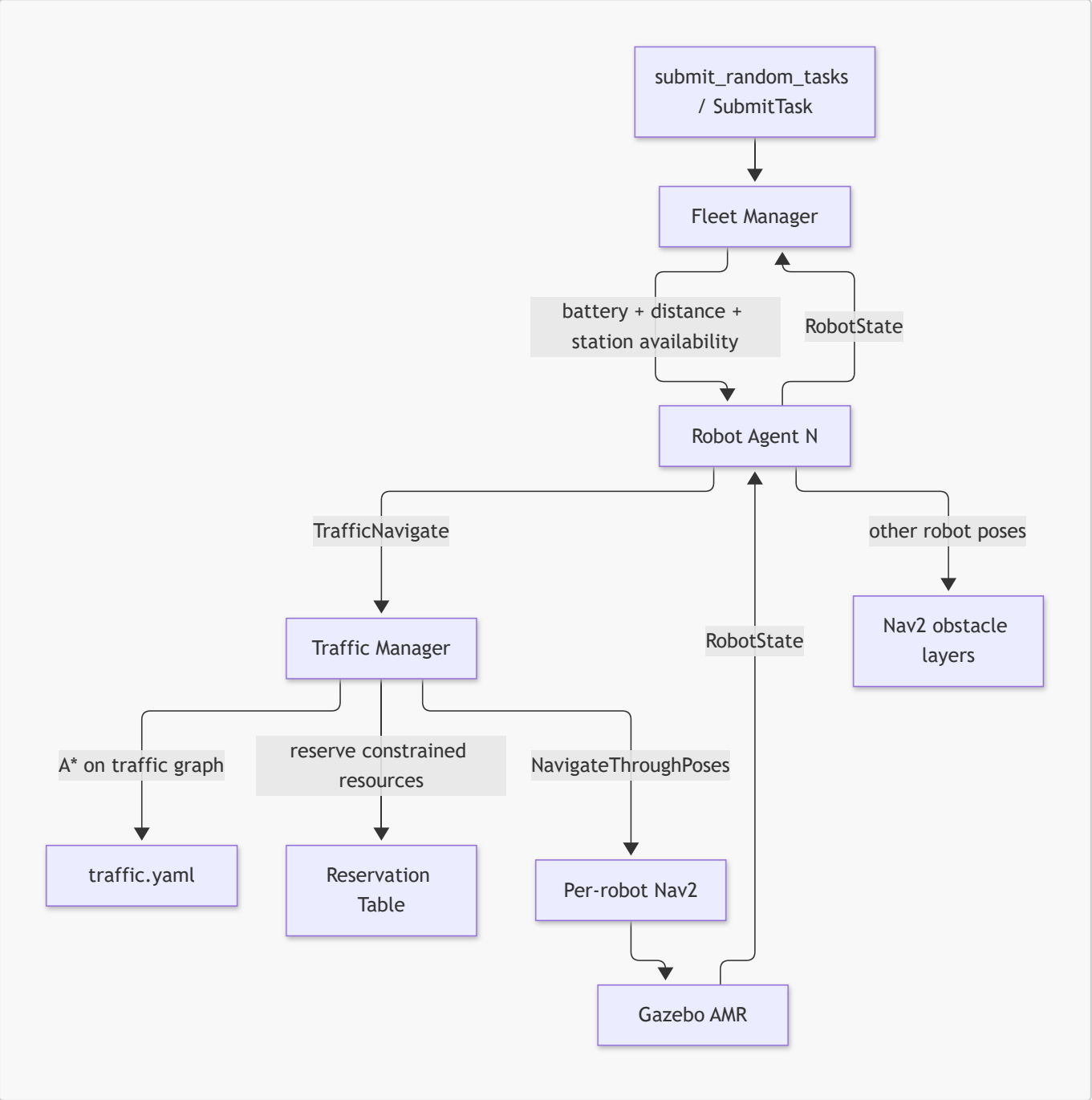
## Repository structure

```
src/
├── warehouse_description/
│   └── urdf/amr.urdf.xacro
│
├── warehouse_gazebo/
│   ├── config/
│   │   ├── stations.yaml
│   │   └── traffic.yaml
│   ├── launch/
│   ├── tools/generate_world.py
│   └── worlds/warehouse.sdf
│
├── warehouse_navigation/
│   ├── config/nav2_params.yaml
│   ├── launch/
│   ├── maps/
│   └── scripts/send_goal.py
│
├── warehouse_msgs/
│   ├── action/
│   │   ├── ExecuteMission.action
│   │   └── TrafficNavigate.action
│   ├── msg/
│   │   ├── FleetStatus.msg
│   │   └── RobotState.msg
│   └── srv/
│       ├── RequestDock.srv
│       └── SubmitTask.srv
│
```

```
├── warehouse_robot_agent/
│   └── warehouse_robot_agent/agent.py
│
└── warehouse_fleet_manager/
    ├── launch/
    └── warehouse_fleet_manager/
        ├── manager.py
        ├── monitor.py
        ├── submit_random_tasks.py
        └── traffic_manager.py
```

Furthermore, the `CMakeLists.txt` and `package.xml` are part of the individual package folders (`warehouse_*`)

## System overview



The architecture separates three responsibilities:

- **Fleet Manager** — task queue, robot selection, station/dock coordination, battery-aware assignment.
- **Traffic Manager** — graph routing, lane/aisle reservations, rerouting around reserved resources, deadlock watchdog, Nav2 `NavigateThroughPoses` execution.
- **Robot Agent** — per-robot mission FSM, battery model, local state, dock return/charging, fleet-state publication, and navigation requests.

# Main capabilities

## Multi-robot simulation

- 1–8 AMRs from one launch configuration.
- Namespaced robot topics, actions, Nav2 stacks, odometry, scans, and robot-state topics.
- Gazebo warehouse world and RViz visualization.
- Differential-drive AMR model generated from Xacro.

## Generated warehouse environment

`warehouse_gazebo/tools/generate_world.py` is the single source for the warehouse layout. It generates:

- `warehouse.sdf`
- `stations.yaml`
- `traffic.yaml`
- ground-truth occupancy map used by AMCL
- navigation occupancy map used by Nav2

The warehouse contains:

- 3 rack rows with 2 rack blocks per row
- traversable horizontal and vertical rack aisles
- 8 charging docks along the south side
- 4 pickup stations on the west side
- 4 drop stations on the east side

## Navigation

Each robot has an independent Nav2 stack using:

- AMCL (Adaptive Monte Carlo Localization)
- generated static map
- NavFn global planner with A* enabled
- Regulated Pure Pursuit controller
- local/global obstacle layers
- robot footprint and inflation layers
- `NavigateThroughPoses` for fleet-approved graph routes

The robot agents also publish other AMR poses as obstacle points so local costmaps can react to nearby robots.

### Fleet task management

Tasks are submitted through:

```
/fleet/submit_task
```

The Fleet Manager:

1. validates pickup/drop station IDs;
2. queues tasks;
3. serializes access to pickup/drop stations;
4. filters robots by freshness, health, state, battery, retries, and availability;
5. estimates mission energy including pickup → drop → nearest dock;
6. selects the eligible robot nearest to the pickup;
7. sends an `ExecuteMission` action to that robot;
8. requeues failed tasks according to retry policy.

### Battery and dock management

Robot agents simulate battery consumption from distance travelled and idle load.

Implemented behavior includes:

- minimum SOC for task assignment;
- mission-energy estimation;
- low-battery mission abort before the robot can no longer reach a dock;
- nearest-free-dock selection;
- automatic idle return to a dock;
- charging state and configurable charge thresholds;
- fault handling when battery is depleted or dock return fails.

### Fleet traffic management

The current traffic manager is a central coordinator between robot agents and Nav2.

For every navigation leg it:

1. obtains the robot's current pose;
2. plans an A* route on the generated warehouse traffic graph;
3. removes resources already reserved by other robots from the candidate graph;
4. converts route edges and internal graph nodes into constrained resources;
5. atomically reserves all required resources for the complete navigation leg;
6. converts the approved graph route into dense waypoints;
7. executes the route with Nav2 `NavigateThroughPoses`;
8. releases the reservations when the leg succeeds, fails, or is cancelled.

Rack aisles, outer corridors, station approaches, dock approaches, and south-lane segments are currently configured as capacity-one resources.

A wait-for graph and cycle detector are included as a deadlock watchdog. When a wait-cycle is detected, the newest waiter yields and replans rather than forcibly releasing a resource underneath a moving robot.

### Random and balanced task generation

Random workload:

```
ros2 run warehouse_fleet_manager submit_random_tasks \
  --count 15 --interval 5 --seed 1
```

Balanced regression workload:

```
ros2 run warehouse_fleet_manager submit_random_tasks \
  --count 16 --interval 5 --seed 1 --balanced
```

With no fixed station and `--balanced`, each block of 16 tasks contains every `pick_1..pick_4 × drop_1..drop_4` combination exactly once, shuffled reproducibly by `--seed`.

A fixed pickup or drop can also be supplied:

```
ros2 run warehouse_fleet_manager submit_random_tasks \
  --pickup pick_1 --drop drop_4 --count 1
```

## Target software stack

- Ubuntu 24.04
- ROS 2 Jazzy
- Gazebo / ROS-Gazebo bridge
- Nav2
- RViz2
- Python 3 / `rclpy`
- `yaml`

The ROS package manifests contain the package-level runtime dependencies used by each component.

## Build

From the workspace root:

```
colcon build --symlink-install --parallel-workers 2
source install/setup.bash
```

If interfaces or Python entry points have changed, make sure the workspace is sourced again before running ROS CLI commands.

## Generate the warehouse files

Regenerate the world, maps, stations, and traffic graph after changing layout geometry:

```
python3 src/warehouse_gazebo/tools/generate_world.py
```

Generated files should not be edited manually; update `generate_world.py` instead.

## Run the simulation

Example with 4 robots:

```
ros2 launch warehouse_fleet_manager fleet_sim.launch.xml \
  num_robots:=4 headless:=true
```

Useful launch arguments include:

```
num_robots
headless
rviz
lidar_rate
lidar_samples
nav_stagger
lifecycle_delay
initial_soc
drain_per_meter
charge_rate
```

For 4 or more robots on a loaded machine, increasing `nav_stagger` and `lifecycle_delay` can make Nav2 lifecycle startup more reliable.

## Monitor the fleet

In a second sourced terminal:

```
ros2 run warehouse_fleet_manager fleet_monitor
```

- Fleet state is also published on `/fleet/status`
- Per-robot state is available on `/robot_N/robot_state`

---

# Submit tasks

Random tasks:

```
ros2 run warehouse_fleet_manager submit_random_tasks \
  --count 10 --interval 5 --seed 1
```

Balanced coverage:

```
ros2 run warehouse_fleet_manager submit_random_tasks \
  --count 16 --interval 5 --seed 1 --balanced
```

Direct service call:

```
ros2 service call /fleet/submit_task warehouse_msgs/srv/SubmitTask \
  "{pickup_id: pick_1, drop_id: drop_3}"
```

# Important ROS interfaces

## Services

```
/fleet/submit_task
/fleet/request_dock
```

## Fleet traffic action

```
/fleet/traffic_navigate
```

## Per-robot mission action

```
/robot_N/execute_mission
```

## Per-robot Nav2 route action

```
/robot_N/navigate_through_poses
```

## Robot state

```
/robot_*N*/robot_state
```

## Current traffic-management strategy

The present implementation prioritizes conflict reduction over maximum throughput.

### What it currently does well

- prevents multiple robots from intentionally reserving the same constrained corridor at the same time;
- reroutes around already-reserved resources when an alternative graph path exists;
- includes rack aisles in normal route planning instead of blocking the whole rack region;
- reserves station and dock access segments;
- reduces free-space diagonal shortcuts by making task endpoints explicit graph nodes;
- keeps reservations during a Nav2 failure/replan so another robot cannot immediately enter a corridor that may still contain the failed robot;
- detects wait-for cycles as a deadlock watchdog.

### Current limitations

- **Whole-leg reservation is conservative.** A robot reserves every constrained resource required for a complete navigation leg before moving.
- This often limits the fleet to only 1–2 simultaneously active missions even when 4 robots are available.
- Intersections and long corridors can remain locked longer than necessary.
- Rerouting currently treats occupied resources primarily as blocked/free rather than using predicted wait-time or congestion cost.
- Head-on conflicts are reduced but not yet guaranteed to be eliminated in every dynamic case.

## Development objective

The project is evolving from independent per-robot Nav2 navigation into a measurable fleet-level coordination platform. The target is not simply to minimize collision count, but to maximize **fleet throughput and concurrent task execution while maintaining safe lane usage**.

## License

Package manifests currently declare Apache-2.0.